

Lifeline: Optimal Byzantine Agreement Under Minimal Synchrony

Yuval Efron^{*1} and Ling Ren^{†2}

¹Columbia University

²University of Illinois Urbana-Champaign

July 18, 2025

Abstract

The synchrony model allows Byzantine Agreement (BA) protocols to be deterministic, tolerate minority faults, and achieve the asymptotically optimal $O(n)$ rounds, and $O(n^2)$ bits of communication where n is the number of parties. We study the deterministic BA problem in a model in which every communication link is either synchronous or partially synchronous. Our main result for this model is that feasibility implies optimality: For every $\frac{n}{3} \leq f < \frac{n}{2}$, the minimal network conditions required for BA to be *solvable* against f byzantine faults, are also sufficient for it to be solvable *optimally*, i.e., with $O(f)$ rounds and $O(f^2)$ communication. In particular, BA against minority byzantine faults can be solved when the synchronous links in the network form a mere *path* (f synchronous links) as efficiently (up to constant factors) as when *all* communication links are synchronous ($\Omega(f^2)$ synchronous links).

1 Introduction

Byzantine Agreement (BA), the problem of bringing n parties, of which up to f are *corrupt* and controlled by an adversary, to agreement on a value, is a cornerstone of distributed computing. The popularity of the problem has surged in recent years due to its paramount importance in the context of blockchain technologies [Nak08], but its legacy is far more extensive, with research on the problem dating back well into the previous century [LSP82].

Perhaps the most fundamental aspect of the model when approaching the BA problem is the *timing model*. The synchrony model assumes that all messages are delivered within Δ time for some known upper bound Δ . As the most optimistic of mainstream timing models, synchrony allows deterministic BA protocols to manifest their apex form, boasting resilience against any minority ($f < n/2$) of byzantine faults, $O(n)$ rounds, and $O(n^2)$ communication [MR21]. The $f < n/2$ resilience is optimal for BA. The round and communication complexities are also optimal when $f = \Omega(n)$ due to the Dolev-Strong and Dolev-Reschuk lower bounds [DS83, DR85].

However, the synchrony assumption is arguably a tad too good to be true: Even a single message being delayed for more than Δ time nullifies any security proof and potentially breaks the protocol. Given the brittleness of synchrony, it is natural to embark on a search for a weaker timing model in which optimal resilience and optimal efficiency are feasible for deterministic BA protocols.

^{*}ye2210@columbia.edu

[†]renling@illinois.edu

One might be tempted to commence this search with asynchrony, the weakest timing model. In asynchrony, messages are delivered after a finite amount of time, at the discretion of the adversary. Alas, any initial hope of designing deterministic BA protocols in asynchrony is quickly evaporated in the face of the seminal FLP impossibility result [FLP85], stating that no deterministic protocol can guarantee termination in asynchrony, even against a single crash fault. The next natural candidate is the partial synchrony model [DLS88]. Over the years, the partial synchrony model established itself as the go-to compromise between the brittleness of synchrony and the impossibility of asynchrony. In partial synchrony, there is a Global Stabilization Time (GST), and the network is asynchronous before GST, and synchronous after GST. Alas, as it turns out, such a weakening of synchrony comes with a price, with the result of [DLS88] showing that protocols in partial synchrony can tolerate at most $f < \frac{n}{3}$ byzantine faults. It thus seems that *some* amount of synchrony is necessary for optimal BA. The question is, how much?

Towards minimal synchrony. On further consideration, one might argue that the landscape of timing models is too *coarse-grained*, especially given the variability and heterogeneity of modern networks. Specifically, in the context of synchrony, a natural question arises.

What is the *minimal* amount of synchrony required for optimal deterministic BA?

In this work, we consider the recently proposed network model of *granular synchrony* [GAC⁺24], which cleanly interpolates between synchrony and partial synchrony. Each communication link is either synchronous or partially synchronous, i.e., it respects the Δ timing bound either at all times or only after GST.

The recent work of [GAC⁺24] established, for each $\frac{n}{3} \leq f < \frac{n}{2}$, tight conditions for the feasibility of BA in granular synchrony, expressed via properties of the graph G_S of synchronous links in the network (See Section 2 for further details). However, their Byzantine agreement algorithms (to establish the feasibility) are far from optimal, using $O(n^2)$ rounds and $O(n^4)$ bits of communication.

1.1 Our contributions

Our main result is that Byzantine agreement can be solved with optimal rounds and communication *whenever it can be solved at all* in the granular synchrony model.

Theorem 1.1 (Informal version of Theorem 5.1). *Let $\frac{n}{3} \leq f < \frac{n}{2}$. There is a protocol, Lifeline, solving BA in $O(n)$ rounds and using $O(n^2)$ bits of communication, in any granular synchrony network in which BA is feasible, and against f byzantine faults.*

To illustrate the strength of our result, let us focus for a moment on the $n = 2f + 1$ case.

BA on a line. It is perhaps not too hard to observe that *full synchrony*, i.e. all communication links being synchronous, is quite the overkill for the feasibility of the BA problem. One can notice that even if not all communication links are synchronous (i.e. G_S isn't a clique), all correct parties being in the same connected component of G_S (e.g., via a simple path of synchronous links) is a sufficient condition for the feasibility of BA, as repeated flooding of messages along this path should allow correct parties to simulate any synchronous protocol, and solve the BA problem. Such a condition turns out to also be necessary for the $n = 2f + 1$ case [GAC⁺24]. In terms of efficiency, however, the price of flooding is quite dire, especially when correct parties *do not know* which

of their links are part of the synchronous path. Our main result in particular establishes that such indiscriminate flooding is not necessary, and BA on a line can be asymptotically solved *as efficiently* as the full synchrony case. In general, for any $\frac{n}{3} \leq f < \frac{n}{2}$, our main result establishes that in granular synchrony, feasibility implies optimality: The minimal network conditions required for deterministic BA to be feasible, also enable optimally efficient protocols.

In the next section, we discuss and put our result in context of related work. Afterwards, in [Section 2](#), we formally define our model and define the primitives used in this work. In [Section 3](#), we give a technical overview of our main result, providing intuition and details both on our construction and the granular synchrony model. [Section 4](#) presents our main result in a simplified setting, allowing us to highlight our technical contributions. [Section 5](#) presents our main result in its full generality.

1.2 Related Work

The seminal results of Dolev-Strong and Dolev-Reschuk [\[DS83, DR85\]](#) establish barriers on the *efficiency* of deterministic BA protocols in *full synchrony*, i.e., when all communication links are synchronous, requiring any purported protocol to take at least $f + 1$ rounds, and use $\Omega(nf)$ bits of communication, respectively. In fact, both bounds hold even against *omission* failures [\[CGG⁺24\]](#), and against very weak validity requirements [\[CGG⁺24\]](#). BA with $O(f)$ round and $O(nf)$ bits of communication was achieved for $f < \frac{n}{3}$ by Berman et al. [\[BGP92\]](#). Perhaps surprisingly, the above efficiency for the optimal $f < \frac{n}{2}$ resilience was achieved only recently in the work of Momose and Ren [\[MR21\]](#). As such, deterministic synchronous BA protocols at their peak form offer resilience against minority ($f < \frac{n}{2}$) byzantine faults, and optimal complexity, i.e., $O(f)$ rounds and $O(f^2)$ communication. We mention for completeness that the above optimal efficiency is also achievable in partial synchrony when $f < \frac{n}{3}$, as shown in the recent work of [\[CDG⁺22\]](#).

Several past works have considered mixed models in which communication links vary from one another in their timing guarantees. A line of works [\[BMR15, HMT07a, ADGFT04\]](#) consider the minimal conditions required to circumvent the FLP impossibility result, by considering a model in which links are either asynchronous or partially synchronous. The work of [\[GAC⁺24\]](#) considers the granular synchrony model we consider in this paper, for which they prove tight conditions for the feasibility of BA when $f \geq \frac{n}{3}$. Their protocols are far from optimal in terms of efficiency however, with $O(n^2)$ and $O(n^4)$ round and communication complexity, respectively.

2 Model & Definitions

Model. We assume a peer-to-peer communication network, with a communication link between each pair of parties i, j among n parties ($V = [n]$ denotes the set of parties). At most f of the parties are corrupt, controlled by an *adversary*. We assume a permissioned setting, i.e., the identities of all parties are known to all parties, and all correct parties are active in the protocol at all times. We assume a Public Key Infrastructure (PKI), in which each party is handed a public/secret key pair (sk_i, pk_i) to be used throughout the protocol. We use the established PKI for digital signatures and threshold signatures. We denote by λ the security parameter dictating the length of signatures/threshold signatures.

We consider the *granular synchrony* model, defined in [\[GAC⁺24\]](#), in which each link is either *synchronous* or *partially synchronous*. A synchronous link delivers each message within a known

upper bound of time Δ , while a partially synchronous link respects the Δ delivery bound only after some unknown point in time, called GST. Prior to GST, the delivery timings of messages on partially synchronous links are controlled by the adversary. We make the following simplifying assumption on clock drift and skew: Let (p, q) be a synchronous link between correct parties, then p, q commence the protocol at the same global time (prior to GST), and have no clock drift between them. Our results can be easily generalized to a case where clock drift and skew between correct parties connected by synchronous links have known upper bounds. Often in the analysis, we refer to a clock keeping *global time*, which is external to the protocol. In particular, the local clocks of correct parties may deviate arbitrarily from the global clock, subject to the above constraint.

Throughout this paper, we always assume that correct parties are *not* aware of the topology of the network, i.e., whether any link is synchronous or partially synchronous. We proceed with some definitions.

Definition 2.1 (Synchronous paths). *Let $G_S = (V, S, F)$ be the subgraph of the network induced by the set S of its synchronous edges, with F being the set of corrupt parties. Let a, b be two parties. We say that a has a synchronous path to party b , and denote $a \rightarrow b$ if there exists a path in G_S from a to b in which all intermediate parties are correct.*

With Definition 2.1 in mind, we define the following relation between two (not necessarily disjoint) sets of parties A, B .

Definition 2.2. *Let A, B be subsets of parties. We say that $A \rightarrow B$ if for all $b \in B$, there exists $a \in A$ s.t. $a \rightarrow b$.*

We can also define the *diameter* of the network to be $d(G_S) = \max_{F, |F| \leq f, a, b} d(a, b)$. Here $d(a, b)$ denotes the length of the shortest path realizing $a \rightarrow b$ in G_S . Note that the diameter could be ∞ .

The paper of [GAC⁺24] provides necessary and sufficient conditions for the feasibility of the BA problem in granular synchrony in the presence of at most f corrupt parties.

Theorem 2.3 (Theorem 7 of [GAC⁺24]). *In the granular synchrony model, BA is solvable deterministically if and only if for all $F \subseteq V, |F| \leq f$, the following holds: For all $A \subseteq V - F$, s.t. $|A| \geq n - 2f$, there exists $B \subseteq V - F, |B| \geq f + 1$ such that $A \rightarrow B$.*

The adversary. In light of the above condition, we want clarify the adversary model we are considering. Given a protocol Π , the adversary $\mathcal{A}$ may observe it, and then choose a set $|F| \leq f$ of parties to corrupt, and their behavior throughout the protocol¹. It then gets to decide, for each communication link between correct parties i, j , whether (i, j) is synchronous, or partially synchronous, so long as the condition in Theorem 2.3 is maintained. The adversary also chooses GST. As mentioned before, prior to GST, the adversary has complete control over message delivery timings throughout the protocol on partially synchronous edges, and it also controls clock drift between correct parties, so long as all correct parties initiate the protocol prior to GST, and correct parties connected by a synchronous link commence the protocol at the same time and have no clock drift w.r.t. one another. Given an adversary $\mathcal{A}$, we denote the graph over synchronous links induced by its behavior by $G_{\mathcal{A}} = (V, S, F)$. We stress that correct parties *do not know* the topology of $G_{\mathcal{A}}$. We refer to an adversary that satisfies the above as an *f-bounded* adversary. More formally:

¹In this work we consider only deterministic protocols, and so w.l.o.g. the behavior of the parties in F is determined by the description of the protocol and the other adversary choices.

Definition 2.4. Let $\frac{n}{3} \leq f < \frac{n}{2}$. Let $\mathcal{A}$ be an adversary and consider the communication graph induced by its behavior $G_{\mathcal{A}} = (V, S, F)$. We say that $\mathcal{A}$ is f -bounded if the following holds.

- $|F| \leq f$
- For all $A \subseteq V - F$ s.t. $|A| \geq n - 2f$, there exists $B \subseteq V - F, |B| \geq f + 1$, s.t. $A \rightarrow B$.

As we are dealing with Byzantine faults, one can assume w.l.o.g. that $|F| = f$, because the adversary can always corrupt a party and have it behave honestly. We now define the Byzantine Agreement (BA) problem.

Definition 2.5 (Byzantine Agreement). Let Π be a protocol executed by parties $p_1, \dots, p_n$, where party p_i has input $m_i \in I$, and produces an output $o_i \in I$. The BA problem pertains to the following properties.

- Π has weak validity if the following holds: If all parties are correct and have the same input m , then all correct parties output m .
- Π has agreement if the following holds: All correct parties that output, output the same value.
- Π is terminating if the following holds: Eventually, all correct parties output and halt.

We say that Π is an f -secure BA protocol if it is valid, has agreement and terminating against any f -bounded adversary.

It is worth mentioning that the protocol we present in this paper also achieves *external validity*, a variant of consensus commonly used in practice. Our protocol also achieves the strong validity property against omission faults: If all parties have the same input m , then all correct parties output m . Note that the $\Omega(f)$ round and $\Omega(f^2)$ communication lower bounds apply even to omission faults or weak validity [CGG⁺24].

Complexity measures. It is common practice in the context of deterministic protocols in the partially synchronous model to measure the communication complexity of a protocol Π as the total number of bits sent by correct parties starting from GST until the last correct party terminates. In other words, communication prior to GST is not counted towards the total communication complexity of Π . Latency/round complexity is treated similarly, with the round complexity of a protocol Π defined to be the time interval between GST and the timestep in which the last correct party halts. The natural justification for not measuring complexity prior to GST is that prior to GST, the network is purely asynchronous, and thus, BA cannot be solved deterministically during this period [FLP85]. In the same spirit, we also define round and communication complexities to only count post GST, despite some of the communication links being synchronous even before GST. In particular, communication occurring on synchronous links prior to GST is not counted toward the communication complexity of the protocol. This is justified due to several results in the literature [BMR11, HMT07b] establishing that BA is solvable in a granular asynchronous² network iff G_P , the graph of synchronous and partially synchronous links, has a connected component of size at least $f + 1$ of correct parties (See Definition 2.1). Note that for all $\frac{n}{3} \leq f < \frac{n}{2} - 1$ ³, an

²In granular asynchrony, each link is either asynchronous, partially synchronous, or synchronous

³In the case of $n = 2f + 1$, all $f + 1$ correct parties are in the same connected component.

f -bounded adversary (See Definition 2.4) does not imply the existence of a connected component of correct parties in $G_{\mathcal{A}}$ of size $f + 1$, so BA is unsolvable deterministically prior to GST. We proceed with formal definitions of communication and round complexity in our model.

Definition 2.6. Let Π be a protocol in the granular partial synchrony model, and let $\mathcal{A}$ be an f -bounded adversary. Furthermore, assume that whenever for all f -bounded adversaries, all correct parties eventually terminate when executing Π . Denote by $t(\mathcal{A})$ the timestep in which the last correct party terminates.

We define the communication complexity of Π w.r.t. $\mathcal{A}$, and denote by $\text{CC}_{\mathcal{A}}(\Pi)$, to be the total number of bits sent by correct parties during the time interval $[GST_{\mathcal{A}}, t(\mathcal{A})]$, where $GST_{\mathcal{A}}$ is the GST chosen by $\mathcal{A}$. We define the latency of Π w.r.t. $\mathcal{A}$, and denote by $L_{\mathcal{A}}(\Pi)$, to be the value $t(\mathcal{A}) - GST_{\mathcal{A}}$.

We define the communication complexity of Π , and denote by $\text{CC}(\Pi)$, to be the maximum communication complexity of Π w.r.t. any f -bounded adversary, i.e. $\max_{\mathcal{A}, \mathcal{A} \text{ is } f\text{-bounded}} \text{CC}_{\mathcal{A}}(\Pi)$. Similarly, we define the latency of Π , and denote by $L(\Pi)$, to be the maximum latency of Π w.r.t. any f -bounded adversary, i.e. $\max_{\mathcal{A}, \mathcal{A} \text{ is } f\text{-bounded}} L_{\mathcal{A}}(\Pi)$.

The work of [GAC⁺24] establishes Theorem 2.3 by designing a BA protocol in granular partial synchrony for any given $f < \frac{n}{2}$. In the worst case, their protocol has communication complexity of $O(d_S \cdot n^3)$ and round complexity of $O(d_S \cdot n)$, where d_S is the diameter of $G_{\mathcal{A}}$. In the worst case, of $d_S = O(n)$, these bounds amount to $O(n^4)$ and $O(n^2)$, a significant gap from the $\Omega(n^2)$ and $\Omega(f)$ lower bounds.

Certifiability. In several parts of our protocol, we leverage our PKI setup to endow primitives with certifiability properties, which intuitively allow correct parties to prove the validity of their outputs. Let I be the space of input values, and let $\Sigma = \{0, 1\}^*$. A *verifier* is a function $\text{Ver} : I \times \Sigma \rightarrow \{0, 1\}$. We refer to the second argument to Ver as the *proof*.

Given the definition of a verifier, we now formally define the following Certified Graded Agreement (CGA) primitive.

Definition 2.7 (Certified Graded Agreement). Let Π be a protocol executed by parties $p_1, \dots, p_n$, where party p_i has input $m_i \in I$, and produces output $(o_i, g_i, \sigma_i) \in (I \cup \{\perp\}) \times \{0, 1\} \times \{0, 1\}^*$. Let Ver be a verifier. We say that an output (o_i, g_i, σ_i) is valid if $\text{Ver}(o_i, \sigma_i) = 1$ or $o_i = \perp$. Π is a certified graded agreement (CGA) protocol if ensures the following properties.

- **Eventually valid:** If all correct parties have the same input value m and all correct parties commence Π post GST and within 2Δ time of one another, then every correct party p_i outputs $(m, 1, \sigma_i)$ and it is valid.
- **Consistent:** If a correct party p_i outputs a valid $(m, 1, \sigma_i)$ for some $m \in I$, then at least $f + 1$ correct parties output a valid $(m, *, \sigma)$. Furthermore, every correct party p_j outputs either $\perp$ or a valid $(m, *, \sigma_j)$. Finally, for any value $m' \neq m$, no party can produce a proof σ such that $\text{Ver}(m', \sigma) = 1$.
- **Justified:** For any value m s.t. at most $n - 2f - 1$ correct parties input m to Π , no party can produce a proof σ such that $\text{Ver}(m, \sigma) = 1$.

- Terminating: *There exists T s.t. within time T all correct parties halt and output.*

We say that Π is an f -secure CGA protocol if there exists a verifier Ver such that w.r.t Ver , Π is eventually valid, consistent, justified, and terminating against any f -bounded adversary.

3 Technical Overview

In this section we give a high level and intuitive description of our approach and main techniques obtaining our main result [Theorem 5.1](#). For simplicity and clarity of exposition, we assume in this technical overview that all correct parties have perfect synchronized local clocks. As it turns out, even with perfectly synchronized local clocks, achieving optimal complexities in granular synchrony requires technical novelty and care, as we are dealing with $f \geq \frac{n}{3}$ Byzantine faults in a non-synchronous network.

Starting point. To effectively communicate the main ideas behind our protocol, let us begin by considering a classic partially synchronous deterministic protocol, secure against $f < \frac{n}{3}$ byzantine faults, namely the Tendermint protocol [\[Buc16\]](#), and explain step by step the chain of ideas and observations required to arrive at our result. Like Tendermint, we employ a trusted Public Key Infrastructure (PKI) setup for idealized signatures and threshold signatures.

Recall that Tendermint adopts a view-based framework, i.e. protocol instructions are organized around time periods referred to as *views*, where in each view, a particular subroutine is executed. Specifically, in Tendermint, each view has a designated leader, who is responsible for bringing the correct parties into agreement via a subroutine involving all-to-all communication. We refer to such views, that follow this leader based approach as *leader views*. To achieve $O(n)$ communication within each leader view, one can use two round trips of leader-to-all, and all-to-leader communication, to create certificates, each of size $n - f$, in support of a value m . Once a correct party observes a level-two certificate (i.e., a valid certificate of certificates), it can confidently decide that value as its output, because at least $n - 2f > f$ correct parties hold a level-one certificate for m , and thus no future certificate of size $n - f$ can ever be created for any value other than m .

Dealing with $\frac{n}{3} \leq f < \frac{n}{2}$. The main challenge of granular synchrony comes with the increased fault threshold of $f \geq n/3$. In this regime, one can no longer rely on the classic quorum intersection argument, so we need another method to handle equivocations.

We recall again the necessary and sufficient synchrony condition, proved by [\[GAC⁺24\]](#), under which BA is solvable in granular partial synchrony when $\frac{n}{3} \leq f < \frac{n}{2}$ (see [Definition 2.4](#)), under a f -bounded adversary $\mathcal{A}$: For every set A of correct parties of size at least $n - 2f$, there exists a set B of correct parties of size at least $f + 1$, such that $A \rightarrow B$, i.e., every party in B is reachable from some party in A via a path of synchronous links composed of correct parties.

To shed a clear light on the main ideas of our construction, one can consider the extreme case of $2f + 1 = n$. Plugging this into [Definition 2.4](#), the condition boils down to *every* correct party having a synchronous path to at least $f + 1$ correct parties. If this simpler constraint is still unintuitive to think about, one can observe that a network topology that minimally (in terms of the number of synchronous links) captures this condition is a *simple path*, i.e., when the graph induced by $\mathcal{G}_{\mathcal{A}}$ on the correct parties is a simple path of length $f + 1$ connecting all correct parties. See [Figure 1](#) for an illustration of feasible networks in the extreme cases of $2f + 1 = n$ and $3f = n$.

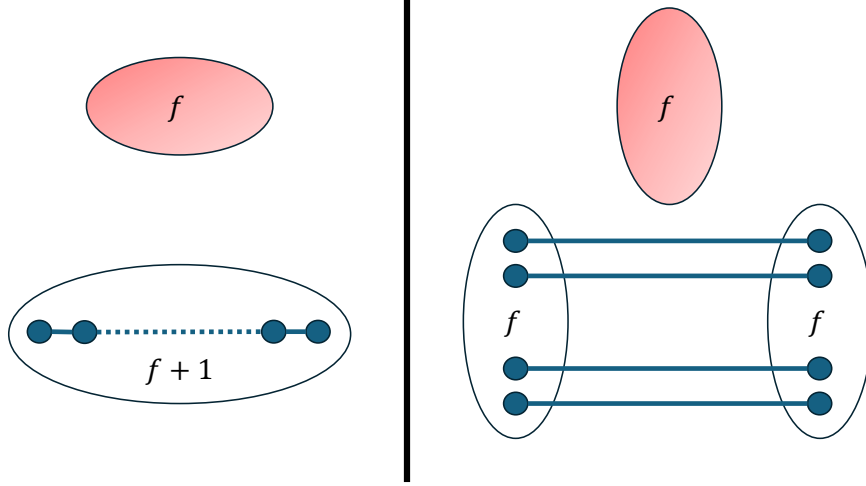


Figure 1: Two extreme examples of minimal network topologies satisfying the synchrony condition of [Definition 2.4](#). On the left, the $n = 2f + 1$ case, in which there are f corrupt parties, depicted by the red oval shape. The $f + 1$ correct parties are connected by a simple path of synchronous links. All omitted links are either partially synchronous or involving a corrupt party. On the right, the $n = 3f$ case for an odd f , the $2f$ correct parties are connected by a matching of synchronous links. Note that any set A of f correct parties has one edge of the matching leaving the set, thus exhibiting a set B of $f + 1$ correct parties s.t. $A \rightarrow B$, as required by [Definition 2.4](#).

We use this special case with one synchronous path to explain our main techniques. We remark that BA with optimal efficiency even for this special case (while assuming synchronized clocks) has remained open so far, and that our final algorithm solves the general case for any feasible network and without synchronized clocks.

First attempt. Perhaps the first idea that comes to mind is to increase the length of every leader view in a Tendermint-style protocol and have correct parties echo certificates along the path to check for equivocations during each leader view. Note that correct parties do not know which of their communication links belongs to the synchronous path. Thus, when forwarding messages, correct parties must forward them to all parties to guarantee usage of the synchronous path. Implemented carefully, this approach can be made to work, and is the engine behind the protocol of [\[GAC⁺24\]](#). However, this approach leads to poor efficiency: To allow correct parties to check for equivocations along the synchronous path, the length of each leader view needs to be $O(n)$ rounds, and the forwarding of certificates (potentially from every party) along the path yields a protocol with round complexity of $O(n^2)$ and $O(n^4)$ communication complexity.

Towards optimal complexity. To achieve better and optimal round and communication complexity, our first idea is to **delegate equivocation detection to the leaders during leader views**. Instead of having all correct parties forward certificates along the synchronous path in each leader view, we let leaders be responsible for detecting equivocation, when they collect the

locked (a la Tendermint) certificates of each party. Roughly, if the leader receives two “up to date” conflicting certificates, it reports those to all parties. A correct party observing an “up to date” equivocation, would relinquish its lock and cooperate with the leader on making a new certificate with the leader’s value of choice. This approach allows us to maintain linear communication and constant rounds per leader view, as parties are not required to forward messages along the synchronous path. See [Section 4.1](#) for further details on the logic of leader views in the synchronized clocks case, and [Section 5.2](#) in the general case.

Accompanying the above delegation of equivocation idea, we can no longer rely on a single honest leader (even post GST) to drive a decision as we cannot trust any single leader to perform the equivocation detection faithfully in its view. Luckily, the first view with a correct leader post GST will still bring the network into a univalent position. Hence, $f + 1$ consecutive leader views post GST are guaranteed to bring the network to a univalent position.

Prior to GST, however, conflicting certificates may avoid detection by correct leaders due to message delays. As such, correct parties face the dilemma of distinguishing between the following two scenarios: Is the network post GST and the network is in a univalent position, or is the network pre GST and equivocations aren’t being detected/progress isn’t being made due to network delays? To resolve this dilemma, we design a novel **Certified Graded Agreement** (CGA) primitive for the granular synchrony model, which we run once every $f + 1$ leader views. Intuitively, the primitive allows each party to break the symmetry in the above dilemma, and either decide a value, or form an opinion (in the form of a value m) moving forward.

Moreover, our CGA has a certifiability property. A party outputting value m from the CGA always has a *proof* in the form of a certificate to justify its output. A crucial property of our CGA is that in each execution, such a proof can exist for at most a single value. As such, our final protocol interlaces a single CGA with $f + 1$ consecutive leader-based views, in repeating fashion, until a decision is reached. Together, they guarantee that once a correct party decides m for some value m , the execution is univalent, and no value other than m can be decided by other correct parties. See [Section 4.2](#) for the full details about our CGA in the synchronized clocks case.

View Synchronization. Lastly, we need to deal with the challenge of clock skew and drift. Towards this end, we need to run a *view synchronization* primitive periodically to guarantee that after GST, correct parties quickly become “caught up” with the most “advanced” correct party, which allows them to reach agreement in a leader view with a correct leader. As observed in [\[CDG⁺22\]](#), though the view logic and view change steps of Tendermint have linear communication complexity, the best view synchronization primitive to our knowledge still involves all-to-all communication. Thus, if view synchronization is run after every view, an adversary can always make the first f leader views have Byzantine leaders, leading to $O(n^3)$ communication. To achieve $O(n^2)$ complexity, we employ a similar approach to the *epoch* based approach of [\[CDG⁺22\]](#), in which leader views are grouped into epochs, each epoch contains $f + 1$ views, and view synchronization is run only once per epoch. Since we are dealing with the $\frac{n}{3} \leq f < \frac{n}{2}$ regime, some modifications to the protocol of [\[CDG⁺22\]](#) are required. See [Section 5.1](#) for the full details.

View synchronization brings new challenges, as it may cause a correct party to abort the current leader view or CGA it is executing, and proceed to a more advanced view/epoch. This complicates the design and analysis of our leader view and CGA primitives. See [Section 5](#) for our full protocol in which both leader views and the CGA primitive are executed concurrently with the view synchronizer Sync.

4 The Lifeline Protocol Assuming Synchronized Clocks

In this section, we present our protocol in a special setting that allows us to highlight our main technical contributions. Specifically, we design an efficient BA protocol in granular partial synchrony assuming that correct parties have *synchronized clocks*. Explicitly, the following holds.

1. All correct parties commence the protocol at the same global time.
2. All correct parties have perfectly synchronized local clocks (i.e., no clock skew or drift). In other words, local time t occurs for all correct parties at the same global time.

This simpler setting allows us to temporarily avoid view synchronization and focus on the main technical ideas of our main Lifeline protocol: The novel CGA protocol, and a low communication leader-based view that detects equivocations. More formally, in this section, we present the following result (which is also new).

Theorem 4.1. *Let $\frac{n}{3} \leq f < \frac{n}{2}$. Assuming PKI, threshold signatures, and synchronized local clocks, there exists a BA protocol $\text{Lifeline}_{\text{sync}}$ that is secure against any f -bounded adversary. Furthermore, $L(\text{Lifeline}_{\text{sync}}) = O(f)$, $CC(\text{Lifeline}_{\text{sync}}) = O(\lambda n^2)$.*

View-based framework. As eluded in the technical overview (See [Section 3](#)), we follow a view-based framework. Roughly, we divide time into intervals, referred to as *views*. The protocol has two types of views, CGA views and leader views. The protocol interlaces $f + 1$ leader views and one CGA view in a repeating fashion until a decision is reached. Transitions between views are dictated by local clock timeouts, which are perfectly synchronized across parties in this section. See [Figure 2](#) for an illustration.

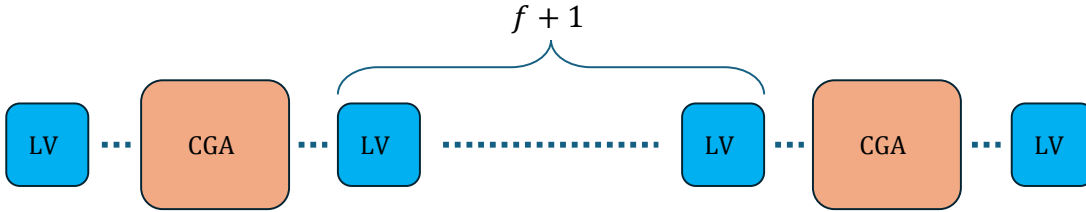


Figure 2: Birds eye view of the structure of $\text{Lifeline}_{\text{sync}}$. The blue boxes represent leader views ([Section 4.1](#)), and the orange boxes represent CGA views ([Section 4.2](#)). The protocol alternates between $f + 1$ leader views with leaders rotating in a round robin fashion, and a single CGA_{sync} view, that can potentially lead to a decision.

Our primary technical tools to establish the above theorem are a CGA protocol, and an efficient leader-based view that handles equivocations. We begin with presenting the leader-based view, and then the CGA protocol, in [Section 4.1](#), and [Section 4.2](#), respectively.

4.1 Leader View with synchronized clocks

In this section, we describe the logic of leader views, which we denote by LV. We indicate by $\text{LV.init}(v)$ that a party commences the execution of a leader view v . During leader views, our protocol intuitively consists of multiple rounds of leader-to-all, all-to-leader communication rounds, in attempt to create certificates of size $n - f$ for values, to then be adopted by correct parties. Notice that unlike partially synchronous protocols, we are in the regime where $f \geq \frac{n}{3}$, so conflicting certificates, each of size $n - f$, may exist. Designing a protocol that detects such equivocations and solves BA while maintaining the *linear* $O(n)$ communication complexity is one of our main technical contributions. We proceed with an intuitive description of the protocol during leader views, before presenting the formal pseudo-code.

Notions. A certificate is a threshold signature by at least $n - f$ parties on the *propose* message for a value. We denote a certificate for a value m by $\mathcal{C}(m, v)$ where v represents the view in which the certificate was created. For a certificate cert , we denote by $\mathbf{m}(\text{cert})$, $\mathbf{v}(\text{cert})$ its content and view, respectively. Concretely, for $\text{cert} = \mathcal{C}(m, v)$, we have $\mathbf{m}(\text{cert}) = m$, $\mathbf{v}(\text{cert}) = v$. Each correct party p_i keeps a local variable cert_i that contains their locked certificate.

The leader view protocol LV. The leader view protocol consists of two round trips of leader-to-all, all-to-leader communication. The protocol commences by having correct parties send to the leader the latest certificate they knew of, held in cert_i . If a party has not observed any certificate, $\text{cert}_i = \perp$, in which case we define $\mathbf{v}(\perp) = -1$, and let $\mathbf{m}(\perp)$ be the input of the given party.

A correct $\text{leader}(v)$ waits Δ time to collect these certificates. If it sees a unique certificate with the highest view, $\text{leader}(v)$ multicasts this certificate to all parties. If it sees conflicting certificates from the highest view, $\text{leader}(v)$ multicasts the two conflicting certificates, along with a proposed value of its choosing. A correct party p_i , upon hearing from $\text{leader}(v)$, locally checks whether the received certificate (or conflicting certificates) is from a higher view than its local cert_i . If so, it updates cert_i . What follows is a single round trip of communication to create a certificate of view v for the proposed value. See Algorithm 4.1 for a more formal protocol description.

The following property is crucial for the overall $\text{Lifeline}_{\text{sync}}$ protocol.

Claim. *Let v be a leader view ($v \neq 0 \pmod{f+2}$). If there exist m and v^* such that (i) at most $n - 2f - 1$ correct parties commence view v with $\mathbf{m}(\text{cert}_i) \neq m$, (ii) any correct party commencing view v with $\mathbf{m}(\text{cert}_i) = m$ has $\mathbf{v}(\text{cert}_i) \geq v^*$, and (iii) there exists no certificate $\mathcal{C}(m', v')$ for any $m' \neq m, v' \geq v^*$, then no certificate $\mathcal{C}(m', v)$ can be created for any $m' \neq m$.*

Proof. Assume towards a contradiction that a certificate $\mathcal{C}(m', v)$ for $m' \neq m$ was created during view v . This implies that at least $n - 2f$ correct parties sent a $(\text{propose}, m', v)$ message to $\text{leader}(v)$. At least one of these parties, call it p_i , commenced view v with $\mathbf{m}(\text{cert}_i) = m$ because at most $n - 2f - 1$ correct parties commenced with certificates for other values. Furthermore, $\mathbf{v}(\text{cert}_i) \geq v'$. For p_i to send $(\text{propose}, m', v)$, one of two cases must hold.

- p_i received a valid opinion_1 message from $\text{leader}(v)$ with value m' . In particular, p_i received certificates $\text{cert}, \text{cert}'$ of the same view but for different messages. Assume, w.l.o.g. that $\mathbf{m}(\text{cert}) \neq m$, then $\mathbf{v}(\text{cert}) \geq \mathbf{v}(\text{cert}_i) \geq v'$, which contradicts the premise of the claim.
- p_i received a valid opinion_2 message from $\text{leader}(v)$ with $\mathbf{m}(\text{cert}) = m'$, and $\mathbf{v}(\text{cert}) \geq \mathbf{v}(\text{cert}_i) \geq v'$, which is again a contradiction.

Algorithm 4.1: Leader view (LV) for party p_i

1. **Upon** LV.init(v):
 - Multicast $\langle \text{commence}, \text{cert}_i, v \rangle$ to leader(v)
 - **If** $p_i = \text{leader}(v)$: wait Δ time. // wait 4Δ without synchronized clocks
2. **Upon** receiving $n - f$ valid commence messages of view v : // only run by leader(v)
 - Let cert be a certificate from the maximum view that leader(v) has seen.
 - **If** leader(v) has also received cert' s.t. $v(\text{cert}') = v(\text{cert})$ and $m(\text{cert}') \neq m(\text{cert})$: multicast $\langle \text{opinion}_1, m, v, \text{cert}, \text{cert}' \rangle$ to all parties where $m = m(\text{cert}_{\text{leader}(v)})$.
 - **Else**: multicast $\langle \text{opinion}_2, \text{cert}, v \rangle$ to all parties.
 - Wait 2Δ time. // wait 4Δ without synchronized clocks
3. **Upon** received valid opinion message from leader(v):
 - **If** valid $\langle \text{opinion}_1, m, v, \text{cert}, \text{cert}' \rangle$ message received with $v(\text{cert}) \geq v(\text{cert}_i)$:
 - Send $\langle \text{propose}, m, v \rangle$ to leader(v)
 - **Elif** valid $\langle \text{opinion}_2, \text{cert}, v \rangle$ message received:
 - If $v(\text{cert}) \geq v(\text{cert}_i)$, set $\text{cert}_i \leftarrow \text{cert}$
 - Send $\langle \text{propose}, m(\text{cert}_i), v \rangle$ to leader(v)
4. **Upon** received $n - f$ propose messages for m of view v : // only run by leader(v)
 - Create $\mathcal{C}(m, v)$ and multicast it to all parties.
5. **Upon** received $\mathcal{C}(m, v)$ from leader(v):
 - $\text{cert}_i \leftarrow \mathcal{C}(m, v)$

In both cases, we arrive at a contradiction. Thus, such a p_i does not exist and no certificate $\mathcal{C}(m', v)$ can be created for $m' \neq m$. $\square$

As for complexity, we prove the following.

Claim 4.2. *Against any f -bounded adversary, LV has $O(\Delta)$ latency and $O(\lambda n)$ communication complexity.*

Proof. The $O(\Delta)$ latency is immediate from the behavior of LV. For communication, note that each correct party sends at most two $O(\lambda)$ -bit messages to leader(v), and leader(v) sends at most two $O(\lambda)$ -bit messages to all parties. $\square$

4.2 Certified Graded Agreement: Synchronized clocks

In this section, we present our CGA for the setting with synchronized clocks. This is the subroutine we execute during CGA views. See Algorithm 4.2 for the formal protocol. Formally, we prove the

following lemma.

Lemma 4.3. *Let $\frac{n}{3} \leq f < \frac{n}{2}$. Assuming PKI, threshold signatures, and synchronized local clocks, there exists a CGA_{sync} protocol Π_{CGA} that is secure against any f -bounded adversary. Furthermore $L(\Pi_{\text{CGA}_{\text{sync}}}) = O(1)$, $\text{CC}(\Pi_{\text{CGA}_{\text{sync}}}) = O(\lambda n^2)$.*

Algorithm 4.2: Certified GA: Synchronized clocks

1. **Upon** $\text{CGA.init}(m_p)$:
 - cert_p is a global variable defined in $\text{Lifeline}_{\text{sync}}$.
 - Start a timer T .
 - Multicast $\langle \text{echo}, m_p \rangle_p$.
2. **During** $0 \leq T \leq \Delta$: If received $n - f$ echo messages for the same value m , create certificate $\mathcal{C}_1(m)$ and multicast $\langle \text{forward}, \mathcal{C}_1(m) \rangle_p$.
3. **During** $\Delta \leq T \leq (2n + 1)\Delta$:
 - If, for the *first* time, received $\langle \text{forward}, \mathcal{C}_1(m) \rangle$, multicast it.
 - If, for the *first* time, received two certificates $\mathcal{C}_1(m), \mathcal{C}_1(m')$ for $m \neq m'$, multicast $\langle \text{forward}, \mathcal{C}_1(m), \mathcal{C}_1(m') \rangle_p$.
4. **Upon** $T = (2n + 1)\Delta$: If created a certificate $\mathcal{C}_1(m)$ in step 2, observed no conflicting certificates, and received $n - f$ forward messages for $\mathcal{C}_1(m)$, then multicast $\langle \text{vote1}, m \rangle_p$.
5. **Upon** $T = (4n + 1)\Delta$: If voted and received $n - f$ vote1 messages for a value m , then create a certificate $\mathcal{C}_2(m)$ of these $n - f$ vote1 messages: set $\text{cert}_p \leftarrow \mathcal{C}_2(m)$ and multicast $\langle \text{vote2}, m, \mathcal{C}_2(m) \rangle_p$.
6. **During** $(4n + 1)\Delta \leq T \leq (6n + 1)\Delta$: If received $\langle \text{vote2}, m, \mathcal{C}_2(m) \rangle$ for the *first* time, multicast it and set $\text{cert}_p \leftarrow \mathcal{C}_2(m)$.
7. **Upon** $T = (8n + 1)\Delta$:
 - If sent vote2 and received $n - f$ vote2 messages for a value m : output $(m, 1, \text{cert}_p)$.
 - Else if received at least one vote2 message for m : set $\text{cert}_p \leftarrow \mathcal{C}_2(m)$ and output $(m, 0, \text{cert}_p)$.
 - Else, output $\perp$.

Connection with LV and $\text{Lifeline}_{\text{sync}}$. When running CGA_{sync} during a CGA view v , a $\mathcal{C}_2$ certificate created during its execution is treated as a valid certificate of view v for the purposes of the leader view protocol LV and the overall $\text{Lifeline}_{\text{sync}}$ protocol. It is saved in a global variable cert_p that is also used in leader view protocol LV instances and the $\text{Lifeline}_{\text{sync}}$ protocol.

Proof of Lemma 4.3. For the purposes of the analysis, the verifier Ver we consider is defined such that $\text{Ver}(m, \sigma) = 1$ iff $\sigma = \mathcal{C}_2(m)$, or $m = \perp$. Let $\frac{n}{3} \leq f < \frac{n}{2}$. Termination is clear, we move to the

other properties. We can also assume there are exactly f Byzantine parties since it never hurts the adversary to corrupt the maximum allowed number of parties in the Byzantine setting.

Claim 4.4. *Against any f -bounded adversary, Π_{CGA} is eventually valid.*

Proof. Let m be a value and assume all correct parties input m to CGA_{sync} . Furthermore, the network is post GST. Note that no correct party multicasts an **echo** message for any value $m' \neq m$. Thus no forward message with certificate $\mathcal{C}_1(m')$ is sent for any $m' \neq m$. Furthermore, as we are post GST and each party waits Δ time prior to proceeding to the next step, all correct parties receive $n - f$ **echo** message for m and create $\mathcal{C}_1(m)$ and **forward** it. Thus, all correct parties multicast a **vote1** message for m and as such they all set $\text{cert} \leftarrow \mathcal{C}_2(m)$ and multicast **vote2** for m . Thus, all correct parties observe $n - f$ **vote2** messages for m , and thus they all output $(m, 1, \mathcal{C}_2(m))$. $\square$

Claim 4.5. *Against any f -bounded adversary, when executing $\Pi_{\text{CGA}_{\text{sync}}}$, a certificate $\mathcal{C}_2(\cdot)$ can be made for at most a single value.*

Proof. Assume towards a contradiction that in some execution of $\Pi_{\text{CGA}_{\text{sync}}}$ against an f -bounded adversary, there exist values $m \neq m'$ s.t. $\mathcal{C}_2(m), \mathcal{C}_2(m')$ are created. That implies that there exist disjoint sets of correct parties S, S' of size at least $n - 2f$ each, such that parties in S multicast a **vote1** message for m , and parties in S' multicast a **vote1** message for m' . This in particular implies that each party in S created a certificate $\mathcal{C}_1(m)$ and multicast it via a **forward** message. Similarly for S' with m' . Furthermore, due to multicasting a **vote1** message, all parties in S forwarded the certificate $\mathcal{C}_1(m)$ created by p without observing a conflicting certificate. The same argument holds for all parties in S' with $\mathcal{C}_1(m')$. By the synchrony condition in Definition 2.4, and the $2n\Delta$ waiting time, we can deduce that no two parties $p \in S, q \in S'$ are in the same connected component of $G_{\mathcal{A}}$. Denoting by $\bar{S}$ and $\bar{S}'$ the connected components of S, S' respectively, we thus have (again from Definition 2.4) that $|\bar{S}| \geq f + 1, |\bar{S}'| \geq f + 1$, both $\bar{S}, \bar{S}'$ consist solely of correct parties, and that $\bar{S} \cap \bar{S}' = \emptyset$. This implies that there are at least $2f + 2$ correct parties. However, this implies that there are $3f + 2$ parties in total, a contradiction to $3f \geq n$. This concludes the proof. $\square$

Claim 4.6. *Against any f -bounded adversary, $\Pi_{\text{CGA}_{\text{sync}}}$ is consistent.*

Proof. Let p be a correct party that outputs $(m, 1, \mathcal{C}_2(m))$. This implies that p observed at least $n - f$ valid **vote2** messages for m , at least $n - 2f$ of which came from correct parties. Denote this set of correct parties by S . This implies that every $p \in S$ has voted **vote2** for m and multicast its certificate $\mathcal{C}_2(m)$. By the behavior of the protocol, specifically step 5, we have that that all correct parties in the connected component of p have received $\mathcal{C}_2(m)$, forwarded it, and set their own local certificate variable to $\mathcal{C}_2(m)$. Since this holds for all $p \in S$, the synchrony condition (Definition 2.4) implies that at least $f + 1$ correct parties set their **cert** variable to $\mathcal{C}_2(m)$ and set $\sigma = \mathcal{C}_2(m)$. And thus at least $f + 1$ correct parties exit the CGA with output $(v, 0, \mathcal{C}_2(m))$. Combined with Claim 4.5, this implies that no correct party outputs a valid $(m', *, \sigma)$ for any $m' \neq m$, and no valid σ can be made for any m' . Thus in particular, all other correct parties output either $(m', *, \sigma)$ or $\perp$. $\square$

Claim 4.7. *Against any f -bounded adversary, $\Pi_{\text{CGA}_{\text{sync}}}$ is justified.*

Proof. Let $m' \neq m$, for a certificate $\mathcal{C}_1(m')$ to be created during the second step of $\Pi_{\text{CGA}_{\text{sync}}}$ by any party, at least $n - f$ **echo** messages must be obtained, which means at least $n - 2f$ of those must originate from correct parties. However, at most $n - 2f - 1$ correct parties send **echo** messages for

Algorithm 4.3: Lifeline_{sync}

Each party p_i maintains the following local variables.

- $ViewLength \leftarrow 4\Delta$
- $GALength \leftarrow 10n\Delta$
- $cert_i \leftarrow \perp$
- $v \leftarrow 0$

While True:

- (a) If $v = 0 \bmod f + 2$, execute $CGA_{sync}.init(cert_i)$ and exit view v after $GALength$ time. If the output has grade 1 (i.e., $(*, 1, *)$), send $\langle \text{decide}, m(cert_i) \rangle$ to all parties.
- (b) Else, execute $LV.init(v)$ and exit view v after $ViewLength$ time.
- (c) In either case, $v \leftarrow v + 1$.

Upon received $f + 1$ decide messages (or certificate of such) for value m :

- Create certificate of $f + 1$ decide messages and multicast it to all parties.
- Output m and terminate.

m' . Thus no $\mathcal{C}_1(m')$ certificate can be created by any party. Thus no correct party casts a **vote1** message for m' , and thus no $\mathcal{C}_2(m')$ certificate can be created. $\square$

Claim 4.8. $\Pi_{CGA_{sync}}$ has communication complexity $O(\lambda n^2)$ and round complexity $O(n) = O(f)$.

Proof. All correct parties terminate $\Pi_{CGA_{sync}}$ in $O(n\Delta)$ time. As we treat $\Delta = O(1)$, we have that the round complexity of $\Pi_{CGA_{sync}}$ is $O(n) = O(f)$, as $f \geq \frac{n}{3}$. As for communication complexity, note that in all steps except for steps 3 and 6 of $\Pi_{CGA_{sync}}$, each party multicasts a single message of at most $O(\lambda)$ bits, where λ describes the length of a signature. For step 3, at most two messages can ever be multicast by each correct party: One for the first $\mathcal{C}_1$ certificate observed, and another on the first occasion of witnessing conflicting $\mathcal{C}_1$ certificates. Thus, in step 3, each correct party also sends messages totaling $O(\lambda)$ bits. In step 6, note that by [Claim 4.5](#), there exists a $\mathcal{C}_2$ for at most a single value m , thus each correct party forwards at most one **vote2** message. We conclude that in all steps of CGA_{sync} , each correct party multicasts $O(\lambda)$ bits. Thus, in total, the communication complexity of $\Pi_{CGA_{sync}}$ is $O(\lambda n^2)$. $\square$

$\square$

4.3 Lifeline: Synchronized clocks

In this section, we prove [Theorem 4.1](#). Let $\frac{n}{3} \leq f < \frac{n}{2}$. The description of Lifeline_{sync} can be found in [Algorithm 4.3](#).

Lemma 4.9. *No two correct parties output different values.*

Proof. Assume towards a contradiction that two correct parties output different values. Since $f+1$ decide messages are required for output, there must exist two correct parties p, q that send decide messages for values $m_p \neq m_q$. Let v_p, v_q be the CGA views that made p, q send their decide messages, respectively. In other words, v_p is the CGA view for which p output $(m_p, 1)$, which we denote by $\text{CGA}[v_p]$, and analogously for v_q . Assume w.l.o.g. that $v_p \leq v_q$. By consistency (Lemma 4.3) of $\text{CGA}[v_p]$, we get that $v_p \neq v_q$. Hence, $v_p < v_q$. We now employ the following claim, whose proof is subsumed by the proof of Claim 5.15.

Claim 4.10. *For any view v $v_p < v \leq v_q$, no valid certificate is created for any value $m \neq m_p$ for view v . Furthermore, at most $n - 2f - 1$ correct parties commence view v with $\mathbf{m}(\text{cert}) \neq m_p$.*

As no certificate of view v_q is created for any other value other than m_p , it has to be that $m_q = m_p$. This contradicts that p, q send decide messages for different values. $\square$

Lemma 4.11. *Against any f -bounded adversary, all correct parties output and terminate at most $O(f)$ time after GST. In other words, $L(\text{Lifeline}_{\text{sync}}) = O(f)$.*

Proof. Let GST be as chosen by $\mathcal{A}$. Let t_S be the starting time of the first leader based view with a correct leader to start after GST. Note that t_S occurs at most $O(f)$ time after GST. If some correct party terminates in the time interval $[t_S, t_S + \text{Viewlength}]$, then all correct parties terminate by $t_S + \text{ViewLength} + \Delta$ and the claim follows. We now prove the following claim. We can assume for the rest of the proof, that no correct party terminates during time $[t_S, t_S + \text{Viewlength}]$.

Claim 4.12. *Let v be a leader view in which $\text{leader}(v)$ is correct and no correct party terminates during view $[t_v, t_v + \text{Viewlength}]$. Then, there exists a value m s.t. all correct parties have $\mathbf{m}(\text{cert}) = m$, $\mathbf{v}(\text{cert}) = v$ by time $t_v + \text{Viewlength}$.*

Proof. We are post GST, and hence the network is synchronous. By the behavior of LV, $\text{leader}(v)$ receives commence messages from all correct parties before sending an opinion message. This implies that no matter whether $\text{leader}(v)$ sends an opinion_1 or opinion_2 message, all correct parties send a propose message to $\text{leader}(v)$ with value m . In turn, again by the Δ waiting time on the leader, all correct parties have sufficient time to send all propose messages to the leader and hear back about $\mathcal{C}(m, v)$. Note that from the perspective of any correct party, it takes at most 4Δ time to receive $\mathcal{C}(m, v)$ from $\text{leader}(v)$ during view v , as in the worst case, a correct party waited 2Δ time to hear an opinion message from $\text{leader}(v)$, and waited at most 2Δ time to hear a level-1 certificate from $\text{leader}(v)$. Thus, setting the leader view length to be $\text{Viewlength} = 4\Delta$ ensures that all correct processes exit view v with $\text{cert} = \mathcal{C}(m, v)$. $\square$

From this point on, the execution is univalent. Formally, as no other certificate of view v for a different value exists, Section 4.1 guarantees that in all future leader views, no certificate for any other value can be made. This implies that in the next CGA view, which occurs at most $O(f)$ time after t_S , all correct parties commence the CGA_{sync} with input m . By post GST validity (See Claim 4.4), we have that all correct parties output $(m, 1, \sigma)$, and thus all correct parties output m and terminate within Δ time afterwards. For round complexity, note that t_S occurs at most $O(f)$ rounds after GST, and the next CGA view after t_S occurs after at most $O(f)$ rounds. The CGA_{sync} protocol has round complexity $O(f)$, and thus all parties terminate $O(f)$ rounds after GST. $\square$

Lemma 4.13. *Assume all parties are correct and have the same input value m , then all parties output m .*

Proof. We establish that when all parties are correct and all have input m , no party p_i can ever have $\mathbf{m}(\text{cert}_i) \neq m$ (recall that $\mathbf{m}(\perp)$ is equal to the input value). We prove this by induction on the views. The induction base case clearly holds due to the fact that at the beginning of time, all parties have $\text{cert} = \perp$, and no certificates for any value $m' \neq m$ exist (inputs are treated as default certificates of view -1).

For the induction step, let v be some view, and assume that for all $v' < v$, no certificate of view v' has been created for any value $m' \neq m$. If v is a **CGA** view, then by [Claim 4.7](#), no certificate for any value other than m can be created during the CGA_{sync} , and in particular no party exits the CGA_{sync} with $\mathbf{m}(\text{cert}) \neq m$ and $\text{cert} \neq \perp$. Similarly, if v is leader view, then by [Section 4.1](#), no certificate for any value other than m can be created during view v , and in particular no party exits the LV execution of view v with $\mathbf{m}(\text{cert}) \neq m$ and $\text{cert} \neq \perp$.

This proves that throughout the execution of the protocol, no certificates for any value other than m are ever created, thus no party ever outputs $(m', 1, \sigma)$ from the **CGA** for any $m' \neq m$, and no correct party outputs $m' \neq m$ in $\text{Lifeline}_{\text{sync}}$. Combined with [Lemma 4.11](#), we conclude that all parties output m , as required. $\square$

Lemma 4.14. $\text{CC}(\text{Lifeline}_{\text{sync}}) = O(\lambda n^2)$.

Proof. [Lemma 4.11](#) established that $\text{L}(\text{Lifeline}_{\text{sync}}) = O(f)$. As such, we can deduce that after GST, each correct party participates in at most $O(1)$ **CGA** views (as the length of each CGA_{sync} is $\Omega(f)$) and at most $O(f)$ leader views, and serves as the leader at most $O(1)$ times. This allows us to upper bound the total amount of bits communicated by each correct party after GST by $O(\lambda n)$, yielding the desired bound. $\square$

[Lemma 4.9](#), [Lemma 4.11](#), [Lemma 4.13](#), and [Lemma 4.14](#) complete the proof of [Theorem 4.1](#).

5 Lifeline

In this section, we prove our main theorem. Recall [Definition 2.4](#) for the definition of an f -bounded adversary.

Theorem 5.1. *Let $\frac{n}{3} \leq f < \frac{n}{2}$. Assuming PKI and threshold signatures, there exists a BA protocol, Lifeline, that is secure against any f -bounded adversary. Furthermore, after GST, $\text{L}(\text{Lifeline}) = O(f)$, $\text{CC}(\text{Lifeline}) = O(\lambda n^2)$.*

View based framework. Our protocol follows the view-based framework. Roughly, time is divided into intervals, referred to as *views*, with each view v being either a leader view, in which case it has a designated leader $\text{leader}(v)$, or a **CGA** view. Slightly different from [Section 4](#), in this section, each view is represented as a tuple of two integers (e, i) . We refer to the first coordinate e as the *epoch* number of the view, and the second coordinate i as the *view* number. We say that two views v, v' are of the same epoch if $v'[0] = v[0]$. We frequently use the order relation $\leq$ on views where for views v, v' , we say $v' \leq v$ if $v'[0] < v[0]$ or if $v'[0] = v[0], v'[1] \leq v[1]$.

Similar to [Section 4](#), we have two types of epochs. The even-numbered epochs are **CGA** epochs, and odd-numbered epochs are *leader* epochs. Epochs alternate between *leader epochs* and *CGA epochs*. Each **CGA** epoch has a single **CGA** view, while each leader epoch have $f + 1$ LV views.

Aside from the leader view LV and certified graded agreement **CGA** primitives we have already introduced, we now need another main component called **View Synchronizer**. This is the protocol

managing the transitions between views for correct parties, and ensures that eventually, all correct parties spend a sufficiently long period of time in the same view whose designated leader is correct.

The LV and CGA primitives are conceptually the same as their simplified versions in [Section 4](#). However, their designs and analysis in this section require additional care, as they now interact with the view synchronizer.

We describe our view synchronizer, denoted `Sync`, in [Section 5.1](#), our leader view protocol LV in [Section 5.2](#), and our CGA in [Section 5.3](#).

5.1 View Synchronizer

We now present a more formal treatment of the view synchronizer. It has the following interface

- `advance(v)`: the party initiates view v .

We say that a party p enters view v at time t if p triggered `advance(v)` at time t . We say that a party p is in view v at all timeslots between the invocation of `advance(v)` and the following `advance(v')` for $v' \neq v$. The requirements from the view synchronizer pertain to a parameter T , and are captured by the following notion of *synchronization time*.

Definition 5.2. *Let `Sync` be a view synchronizer protocol, and let $\mathcal{A}$ be an adversary. We say that $t_s(\mathcal{A})$ is a synchronization time w.r.t. `Sync` and $\mathcal{A}$ if $t_s(\mathcal{A})$ is a time slot for which the following holds when executing `Sync` with $\mathcal{A}$ as the adversary.*

1. *All correct parties are in the same view v from time $t_s(\mathcal{A})$ to at least time $t_s(\mathcal{A}) + T$.*
2. *`leader(v)` is correct.*

We use a view synchronizer protocol similar to the one in [\[CDG⁺22\]](#). In our setting, however, $f \geq \frac{n}{3}$, so a modification is required. We group views into *epochs*, each consisting of $f + 1$ views. More precisely, for an integer e , an epoch e refers to a set of views $\{(e, 0), \dots, (e, k)\}$ for some k . In our protocol, even-numbered epochs have a single CGA view, and are called CGA epochs, while odd-numbered epochs have $f + 1$ leader views, and are called leader epochs. Transitions between leader views within the same epoch occur without communication and depend solely on local time. Transitions between epochs require all-to-all communication between parties, and such communication only occurs at the end of an epoch. There are several properties we require from our view synchronizer; see [Theorem 5.3](#), and [Claim 5.4](#). The proofs of correctness and complexity for [Theorem 5.3](#) can be found in [\[CDG⁺22\]](#), and they hold even for our modified version of the protocol. We include the full protocol for completeness. For the purposes of exposition, we provide a more textual and succinct description of the synchronizer protocol (`Sync(T)`) in [Algorithm 5.1](#). Below, all references to time refer to the local clock of p_i .

In the below protocol, $\mathcal{C}(\cdot)$ denotes the functionality of a threshold signature scheme, computing a certificate signature from a given set of shares.

The paper of [\[CDG⁺22\]](#) proves the following theorem about the above protocol.

Theorem 5.3 (Theorem 2 in [\[CDG⁺22\]](#), and claims within). *Let $\frac{n}{3} \leq f < \frac{n}{2}$, then `Sync` is a partially synchronous view synchronizer with $\text{CC}(\text{Sync}) = O(\lambda n^2)$ and $L(\text{Sync}) = O(f)$ against any f -bounded adversary. Furthermore, post the first synchronization time, correct parties enter every view within at most 2Δ time of one another. In particular, if t_s is the first synchronization time, occurring at view v , then every subsequent view in which `leader(v)` is correct also has a synchronization time.*

Algorithm 5.1: Sync(T)

Odd numbered epochs are *leader epochs*, and even numbered epochs are *CGA epochs*. Each leader epoch consists of $f + 1$ views, each of which has length $ViewLength$. CGA epochs consist of a single view, that has length $GaLength$. When $v[0] = 1 \bmod 2$, i.e., in leader based epochs, we define $leader(v) = \frac{v[0]-1}{2} \cdot (f + 1) + v[1] \bmod n$. In other words, leaders are rotating in a round robin fashion.

Initialization. At time $t = 0$, p_i initializes the following variables.

- $ViewLength = T + 2\Delta$. This variable describes the amount of time p_i spends in a given leader view, during leader epochs.
 - $GaLength = 50n\Delta$. This variable describes the amount of time p_i spends in a CGA epoch.
 - $view_i \leftarrow (0, 0)$, $epoch_i \leftarrow 0$. Initialize epoch and view numbers.
 - $EpochSig_i \leftarrow \perp$. Used to store a certificate indicating that $n - f$ parties moved to a given epoch.
1. At the beginning of time $t = 0$, p_i initiates epoch 0 ($\text{advance}(0, 0)$).
 2. **In a leader epoch, upon exiting a view** $view_i$, if $view_i[1] \neq f \bmod f + 1$, then $view_i[1] \leftarrow view_i[1] + 1$ and p_i enters view $view_i$ ($\text{advance}(view_i)$). Otherwise p_i indicates that it is done with the current epoch by multicasting $\langle \text{Epoch completed}, epoch_i \rangle_i$. **In a CGA epoch, upon exiting the CGA**, p_i indicates that it is done with the current epoch by multicasting $\langle \text{Epoch completed}, epoch_i \rangle_i$.
 3. **Upon** receiving $\langle \text{Epoch completed}, e \rangle$ for $e \geq epoch_i$ from at least $n - f$ parties, p_i prepares to enter a new epoch by:
 - Updating its certificate: $EpochSig_i \leftarrow \mathcal{C}(\langle \text{Epoch completed}, e \rangle)$.
 - Updating $epoch_i \leftarrow e + 1$.
 - Wait Δ time and then multicast $\langle \text{Enter epoch}, epoch_i, EpochSig \rangle$, enter epoch $epoch_i + 1$, and set $view_i \leftarrow (epoch_i, 0)$.
 4. **Upon** receiving a valid $\langle \text{Enter epoch}, e', EpochSig \rangle_j$ message for an epoch $e' > epoch_i$, p_i does:
 - Updates its certificate $EpochSig_i \leftarrow EpochSig$.
 - $epoch_i \leftarrow e'$.
 - Wait Δ time and then multicast $\langle \text{Enter epoch}, epoch_i, EpochSig \rangle_i$, enter epoch $epoch_i$ ($\text{advance}(epoch_i, 0)$), and set $view_i \leftarrow (epoch_i, 0)$.

Above, the latency of **Sync** refers to the amount of time post GST until the first synchronization time, denoted by t_s . The communication complexity of **Sync** is defined to be the amount of bits sent by correct parties between GST and t_s . In granular partial synchrony, for the correctness of our protocol, we require also a significantly weaker synchronization property to hold also *before* GST, that intuitively states that parties in the same connected component of $G_{\mathcal{A}}$, for every f -bounded adversary $\mathcal{A}$, are somewhat synchronized. Luckily, **Sync** provides this property as well. Formally, we use the following.

Claim 5.4. *Let $f < \frac{n}{2}$, and let $\mathcal{A}$ be an f -bounded adversary (See [Definition 2.4](#)). Finally, let u, w be two correct parties in the same connected component of $G_{\mathcal{A}}$. Then at all times (even before GST), **Sync** guarantees that if u enters a view v , then within $2n\Delta$ time, w enters a view $v' \geq v$.⁴*

Proof. When u entered epoch e , it sent an Enter epoch message for epoch e at some time t . Let $u, a_1, \dots, a_m, w$ be a path in $G_{\mathcal{A}}$ between u, w . Note that clearly $m \leq n - 1$. It suffices to prove that in time at most $t + 2\Delta n$, w enters an epoch $e' \geq e$. The claim follows by induction on m . If $m = 0$, i.e. (u, w) is a link in $G_{\mathcal{A}}$, then clearly by the behavior of **Sync** and the synchrony of the link, within 2Δ time w either enters epoch e , or has already entered a epoch $e' > e$. For the inductive step, we have that a_1 either enters epoch e within 2Δ time, or has entered *previously* an epoch $e' \geq e$, in which case the inductive hypothesis (on a_1 and w) yields that at most $2m\Delta$ time from that point, w has entered an epoch $e' \geq e$. Thus at most $2(m + 1)\Delta$ time after u has entered epoch e , w has entered an epoch $e' \geq e$. The proof is then concluded by again noticing that $m \leq n - 1$. $\square$

5.2 Leader view

The LV protocol is almost identical as the one in [Section 4](#) where we assumed synchronized clocks. There are only two differences. First, parties run the view core protocol for at most the allotted amount of time, as per the view synchronizer **Sync**. Note that a party may exit a view execution (whether a leader view or CGA view) earlier than the allotted time limit, due to e.g. an “Enter epoch” message for a higher epoch (See [Section 5.1](#)). The other difference is that the two waiting periods by the leader now need to be increased to account for clock skews across parties; see comments in the pseudocode of [Algorithm 4.1](#).

The formal properties we require of LV are highly linked to the overall execution of the protocol, and the interaction between LV and **Sync**. As such, we prove some of the required properties of LV in the analysis of the overall protocol. Specifically, see [Claim 5.15](#), and [Claim 5.17](#).

Here, we prove the following safety and efficiency properties. They are largely the same as those in [Section 4](#). We repeat them for completeness.

Claim 5.5. *Let $\mathcal{A}$ be an f -bounded adversary, let v be a leader view ($v[0] \neq 0 \bmod 2$). Assume there exist m and v^* such that (i) at most $n - 2f - 1$ correct parties commence view v with $m(\text{cert}_i) \neq m$, (ii) any correct party commencing view v with $m(\text{cert}_i) = m$ has $v(\text{cert}_i) \geq v^*$, and (iii) there exists no certificate $\mathcal{C}(m', v')$ for any $m' \neq m, v' \geq v^*$. Then, when executed concurrently with the view synchronizer **Sync**, no certificate $\mathcal{C}(m', v)$ can be created for any $m' \neq m$.*

Proof. Let $m' \neq m$. Assume towards a contradiction that a certificate $\mathcal{C}(m', v)$ was created during view v . This implies that at least $n - 2f$ correct parties sent a $(\text{propose}, m', v)$ to $\text{leader}(v)$. At least

⁴Recall that correct parties in the same connected component of $G_{\mathcal{A}}$ have no clock drift or clock skew w.r.t. one another.

one of these parties, call it p_i , commenced view v with $m(\text{cert}_i) = m$ because at most $n - 2f - 1$ correct parties commenced with certificates for other values. Furthermore, $v(\text{cert}_i) \geq v'$. For p_i to send $(\text{propose}, m', v)$, one of two cases must hold.

- p_i received a valid opinion_1 message from $\text{leader}(v)$ with value m' . In particular, p_i received certificated $\text{cert}, \text{cert}'$ of the same view but for different messages. Assumes w.l.o.g. that $m(\text{cert}) \neq m$, then $v(\text{cert}) \geq v(\text{cert}_i) \geq v'$, which contradicts the premise of the claim.
- p_i received a valid opinion_2 message from $\text{leader}(v)$ with $m(\text{cert}) = m'$, and $v(\text{cert}) \geq v(\text{cert}_i) \geq v'$, which is again a contradiction.

In both cases, we arrive at a contradiction. Thus, such a p does not exist and no certificate $\mathcal{C}(m', v)$ can be created in view v for $m' \neq m$. $\square$

Claim 5.6. *Let v be a leader view in a leader based epoch. The latency of view v is $O(\Delta)$.*

Proof. The claim is straightforward from the behavior of LV. LV has a constant number of steps, in between each correct party waits at most $O(\Delta)$ time. $\square$

Claim 5.7. *Let v be a leader view in a leader based epoch. The total amount of bits communicated by correct parties is $O(\lambda n)$.*

Proof. Each correct non-leader party communicates only with $\text{leader}(v)$, and sends to it a constant number of messages throughout LV, each of size $O(\lambda)$. $\text{leader}(v)$ is the only party to multicast messages to all parties, and it does so for a constant number of messages, each of size $O(\lambda)$. Thus, in total, correct parties use at most $O(\lambda n)$ bits of communication when executing LV. $\square$

5.3 Certified Graded Agreement

The last building block is a certified graded agreement protocol for the granular synchrony model, that boasts a *certifiability* property, as depicted in [Definition 2.7](#). Each execution CGA has a view associated with it as assigned by the view synchronizer protocol, and each message sent by a correct party throughout the protocol also contains the current view identifier. For simplicity of notation, we omit the view identifier from the following description of the protocol. The interface of our CGA includes the `init` functionality, for which $\text{CGA.init}(v)$ for a value $m \in I$ commences an execution of CGA with input value m . Formally, we prove the following.

Lemma 5.8. *Let $f < \frac{n}{2}$, there exists a CGA protocol Π_{CGA} which is secure against any f -bounded adversary when executed concurrently with the view synchronizer `Sync`. Furthermore, Π_{CGA} has communication complexity $O(\lambda n^2)$, and round complexity $O(n) = O(f)$.*

To assist the reader, below are the precise properties we require from our CGA. We prove that the protocol we design for the CGA satisfies the following, when executed concurrently with the view synchronizer `Sync`. See [Algorithm 5.2](#) for the formal protocol description.

- *Eventually Valid:* After GST, if all correct parties commence a CGA epoch within 2Δ time of one another with the same input value m , then all parties output $(m, 1, *)$. See [Claim 5.9](#).

- *Consistent*: If a correct party outputs $(m, 1, *)$ from a CGA, then at least $f + 1$ correct parties exits the CGA view (and epoch) with a $\mathcal{C}_2$ or $\mathcal{C}_3$ certificate for m , i.e., having $v(\text{cert}) = (e, 0)$, $m(\text{cert}) = m$. In other words, they output $(m, 0)$ from the CGA. Moreover, all the other correct parties either exit the CGA with a $\mathcal{C}_2$ or $\mathcal{C}_3$ certificate for m , or output $\perp$. Finally, no $\mathcal{C}_2$ certificate, and in particular $\mathcal{C}_3$ certificate, can be created for any other value m' . See [Claim 5.11](#) and [Claim 5.10](#).
- *Justified*: If for some value m , at most $n - 2f - 1$ correct parties commence the CGA with an input value different than m , then during the CGA execution, a $\mathcal{C}_2$ or $\mathcal{C}_3$ certificate can only be created for m . See [Claim 5.12](#).
- *Terminating*: All correct parties terminate CGA by time $T = GST + 25n\Delta$.

Connection with LV and Lifeline. When running CGA during a CGA view $v = (e, 0)$, $\mathcal{C}_2, \mathcal{C}_3$ certificates created during its execution are treated as valid certificates of view $(e, 0)$ for the purposes of the leader view protocol LV and the overall Lifeline protocol.

Proof. The protocol commences for a party p when it initiates $\text{CGA.init}(v)$ for a value m . The protocol Π_{CGA} is given in Algorithm 5.2 for a given correct party p with input m .

Now, for the analysis. termination is clear. For the purposes of the analysis, as per [Definition 2.7](#), for the CGA executed at view v our verifier Ver of choice is $\text{Ver}(m, \sigma) = 1$ iff σ is a valid $\mathcal{C}_2$ or $\mathcal{C}_3$ certificate for m from Π , or $m = \perp$. Let $\frac{n}{3} \leq f < \frac{n}{2}$.

Claim 5.9. Π_{CGA} is eventually valid against any f -bounded adversary.

Proof. Let m be a value in I and assume all parties input m to CGA and furthermore, the network is post GST and all correct parties commence CGA within 2Δ time of one another. Note that no correct parties multicasts an **echo** message for any value $m' \neq m$, Thus no forward message with certificate $\mathcal{C}_1(m')$ is sent for any $m' \neq m$. Furthermore, as we are post GST and each party waits 4Δ time prior to proceeding to the next step, all correct parties receive $n - f$ **echo** message for m and create $\mathcal{C}_1(m)$ and **forward** it. Thus all correct parties multicast a **vote1** message for m and as such they all set $\text{cert} \leftarrow \mathcal{C}_2(m), \sigma \leftarrow \mathcal{C}_2(m)$ and multicast **vote2** for m . Similarly, all correct parties multicast **vote3** for m and thus all correct parties observe $n - f$ **vote3** messages for m , and thus they all output $(m, 1, \mathcal{C}_3(m))$. $\square$

Before we tend to consistency, we prove the following claim.

Claim 5.10. Against any f -bounded adversary, when executing Π_{CGA} concurrently with the view synchronizer **Sync**, a certificate $\mathcal{C}_2(\cdot)$ can be made for at most a single value $m \in I$.

Proof. Assume towards a contradiction that in some execution of Π_{CGA} (i.e. in a particular CGA epoch) against an f -bounded adversary, there exists values $m \neq m'$ s.t. $\mathcal{C}_2(m), \mathcal{C}_2(m')$ are created. That implies that there exists disjoint sets of correct parties S, S' of size at least $n - 2f$ each, such that parties in S multicast a **vote1** message for m , and parties in S' multicast a **vote1** message for m' . This in particular implies that each party in S created a certificate $\mathcal{C}_1(m)$ and multicast it via a **forward** message. Similarly for S' with m' . By the synchrony condition in [Definition 2.4](#), the $6n\Delta$ waiting time, and [Claim 5.4](#), for every party $p \in S$, all parties in the connected component of p in $G_{\mathcal{A}}$ have been participating in the same CGA instance, as p during the time interval $[p_{\text{forward}} +$

Algorithm 5.2: Certified GA

1. **Upon** $\text{CGA.init}(m_p)$:
 - cert_p is a global variable defined in Lifeline.
 - Start a timer T .
 - Multicast $\langle \text{echo}, m_p \rangle_p$.
2. **Upon** $T = 4\Delta$:
 - If received $n - f$ **echo** messages for a single value m , create certificate $\mathcal{C}_1(m)$, and multicast $\langle \text{forward}, \mathcal{C}_1(m) \rangle_p$.
3. **During** $4\Delta \leq T \leq 6n\Delta + 4\Delta$:
 - If, for the *first* time, received $\langle \text{forward}, \mathcal{C}_1(m) \rangle$, multicast it.
 - If, for the *first* time, received two certificates $\mathcal{C}_1(m), \mathcal{C}_1(m')$ for $m \neq m'$, multicast $\langle \text{forward}, \mathcal{C}_1(m), \mathcal{C}_1(m') \rangle_p$.
4. **Upon** $T = 6n\Delta + 4\Delta$:
 - If created a certificate $\mathcal{C}_1(m)$ in step 2, and observed no conflicting certificates, and received $n - f$ **forward** messages for $\mathcal{C}_1(m)$, multicast $\langle \text{vote1}, m \rangle_p$.
5. **Upon** $T = 12n\Delta + 4\Delta$:
 - If voted, and received $n - f$ **vote1** messages for a value m , create certificate $\mathcal{C}_2(m)$ of **vote1** messages, set $\text{cert}_p \leftarrow \mathcal{C}_2(m)$ and multicast $\langle \text{vote2}, m, \mathcal{C}_2(m) \rangle_p$.
6. **During** $12n\Delta + 4\Delta \leq T \leq 18n\Delta + 4\Delta$:
 - If, for the *first* time, received valid $\langle \text{vote2}, m, \mathcal{C}_2(m) \rangle$ message, multicast it and set $\text{cert}_p \leftarrow \mathcal{C}_2(m)$.
7. **Upon** $T = 18n\Delta + 4\Delta$:
 - If voted and received $n - f$ **vote2** messages for a value m , create certificate $\mathcal{C}_3(m)$ of **vote2** messages, set $\text{cert}_p \leftarrow \mathcal{C}_3(m)$, and multicast $\langle \text{vote3}, m, \mathcal{C}_3(m) \rangle_p$.
8. **Upon** $T = 24n\Delta + 4\Delta$:
 - If voted(**vote3**) and received $n - f$ valid **vote3** messages for m , output $(m, 1, \text{cert}_p)$.
 - Else, if $\text{cert}_p = \mathcal{C}_2(m)$ or $\text{cert}_p = \mathcal{C}_3(m)$ for some m , output $(m, 0, \text{cert}_p)$.
 - Else, output $\perp$.

$2n\Delta - 4\Delta, p_{\text{vote1}} - 2n\Delta]$, where p_{forward} is the time in which p multicasts its first forward message, and p_{vote1} is the time in which p multicasts its `vote1` message. This holds since [Claim 5.4](#) stipulates that epoch changes between correct parties within the same connected component of $G_{\mathcal{A}}$ occur within $2n\Delta$ time steps of one another, and thus if any party in the connected component of p in $G_{\mathcal{A}}$ were to move on (to a higher epoch) from the current CGA epoch during the interval $[p_{\text{forward}} + 2n\Delta - 4\Delta, p_{\text{vote1}} - 2n\Delta]$, p would have moved on as well, and thus it would not have sent a `vote1` message. In particular, [Claim 5.4](#) implies that if two parties in the same connected component are executing the same CGA, then they are at most $2n\Delta$ time apart in their execution logic. This implies that all correct parties in the connected component of p in $G_{\mathcal{A}}$ simultaneously participated in the `forward` phase of the CGA for at least $2n\Delta$ time steps. Thus, they all forwarded the certificate $\mathcal{C}_1(m)$ created by p without creating themselves a conflicting certificate (e.g. for m'). The same argument holds for all parties in S' with $\mathcal{C}_1(m')$. we can thus deduce that no two parties $p \in S, q \in S'$ are in the same connected component of $G_{\mathcal{A}}$. Denoting by $\bar{S}$ and $\bar{S}'$ the connected components of S, S' respectively, we thus have (again from [Definition 2.4](#)) that $|\bar{S}| \geq f + 1, |\bar{S}'| \geq f + 1$, both $\bar{S}, \bar{S}'$ consist solely of correct parties, and that $\bar{S} \cap \bar{S}' = \emptyset$. This implies that there are at least $2f + 2$ correct parties. However, this implies that there are $3f + 2$ parties in total, a contradiction since $3f \geq n$. This concludes the proof. $\square$

Claim 5.11. Π_{CGA} is consistent against any f -bounded adversary, when executed concurrently with the view synchronizer `Sync`.

Proof. Let p be a correct party that outputs $(v, 1, \sigma)$. This implies that p observed at least $n - f$ valid `vote3` messages for v , at least $n - 2f$ of which came from correct parties. Denote this set of correct parties by S . This implies that every $p \in S$ has voted `vote2` for v and multicast its certificate $\mathcal{C}_2(v)$. Furthermore, [Claim 5.4](#) implies that two parties in the same connected component participating in the same CGA instance are at most $2n\Delta$ time apart in their execution logic. Combined with the fact that p multicast a `vote3` message for v , we can deduce that all correct parties in the connected component of p in $G_{\mathcal{A}}$ simultaneously participated in step 6 of the current CGA execution during the time interval $[p_{\text{vote2}} + 2n\Delta, p_{\text{vote3}} - 2n\Delta]$, where p_{vote2} is the timestep in which p multicasts its `vote2`, and p_{vote3} is the timestep in which p multicasts its `vote3` message. This implies that all correct parties in the connected component of p have received $\mathcal{C}_2(v)$, forwarded it, and set their own local certificate variable to $\mathcal{C}_2(v)$. Since this holds for all $p \in S$, the synchrony condition ([Definition 2.4](#)) implies that at least $f + 1$ correct parties set their `cert` variable to $\mathcal{C}_2(v)$. By [Claim 5.10](#), this can only later change to $\mathcal{C}_3(v)$ during the current CGA execution, and thus at least $f + 1$ correct parties exit the CGA with output $(v, 0, \sigma)$

Combined with [Claim 5.10](#), this implies no correct party outputs $(m', *, \sigma)$ for any $m \neq m'$. $\square$

Claim 5.12. Against any f -bounded adversary, Π_{CGA} is justified.

Proof. Let $m' \neq m$, for a certificate $\mathcal{C}_1(m')$ to be created during the second step of Π_{CGA} by any party, at least $n - f$ echo messages must be obtained, which means at least $n - 2f$ of those must originated from correct parties. However, at most $n - 2f - 1$ correct parties sent echo messages for m' . Thus no $\mathcal{C}_1(m')$ certificate can be created by any party. Thus no correct party casts a `vote1` message for m' , and thus no $\mathcal{C}_2(m')$ (or $\mathcal{C}_3(m')$) certificate can be created. $\square$

Now, we analyze the complexity of Π_{CGA} .

Claim 5.13. Π_{CGA} has communication complexity $O(\lambda n^2)$ and round complexity $O(n) = O(f)$.

Proof. All correct parties terminate CGA after at most $36n\Delta + 4\Delta$ rounds, as we treat $\Delta = O(1)$, we have that the round complexity of Π_{CGA} is $O(n) = O(f)$, as $f \geq \frac{n}{3}$. As for communication complexity, note that in all steps except for steps 3 and 6 of CGA, each party multicasts a single message of at most $O(\lambda)$ bits, where λ describes the length of a signature. For step 3, notice at most 2 messages can ever be multicast by each correct party: One for the first $\mathcal{C}_1$ certificate to be observed, and another one on the first occasion of witnessing a conflicting $\mathcal{C}_1$ certificate, thus also in step 3, each correct party multicasts messages totaling $O(\lambda)$ bits. In step 6, note that by Claim 5.10, there exists a $\mathcal{C}_2$ for at most a single value m , thus each correct party forwards a `vote2` message with a $\mathcal{C}_2$ certificate at most once. We conclude that in all steps of the CGA, each correct party multicasts $O(\lambda)$ bits. Thus, in total, the communication complexity of Π_{CGA} is $O(\lambda n^2)$. $\square$

This concludes the proof of Lemma 5.8. $\square$

5.4 Lifeline protocol

In this section, we present our BA protocol Lifeline, the protocol evincing Theorem 5.1. In particular, for a given f , Lifeline is secure against any f -bounded adversary. Instructions described in the following protocol take precedence and are executed before other triggered actions in either LV or Sync. For the purposes of the overarching BA protocol Lifeline, $\mathcal{C}_2(\cdot), \mathcal{C}_3(\cdot)$ certificates created during CGA are treated as valid certificates from the view of any given CGA.

Algorithm 5.3: Lifeline

Data: Each party p_i maintains the following local variables.

- $T \leftarrow 20\Delta$
- $\text{cert}_i \leftarrow \perp$
- $\text{decided} \leftarrow \text{false}$

1. **Upon** beginning of protocol(input): Sync.init(T)
2. **Upon** Sync.advance(view_i): If $\text{view}_i[0] = 1 \bmod 2$, LV.init(view_i).
Else, CGA.init($\text{view}_i, \text{m}(\text{cert}_i)$).
3. **Upon** exiting view view_i :
 - If $\text{view}_i[0] = 0 \bmod 2$ and $\text{decided} = \text{false}$ and output with grade 1 (i.e. $(*, 1, *)$) from CGA of view view_i : Multicast $\langle \text{decide}, \text{m}(\text{cert}_i) \rangle$ to all parties, and set $\text{decided} \leftarrow \text{true}$.
4. **Upon** received $f + 1$ decide messages for value m :
 - Create certificate of $f + 1$ decide messages and multicast it to all parties.
 - decide m and terminate.

5.5 Correctness and complexity

The goal of this section is to prove [Theorem 5.1](#). The proof of [Theorem 5.1](#) is divided into several lemmas, one for each required property.

Lemma 5.14. *Let S be the set of values m for which there exists a correct party that send $\langle \text{decided}, m \rangle$ to all parties. Then $|S| \leq 1$. In particular, all correct parties that terminate, terminate in agreement.*

Proof. Assume towards a contradiction that there exist two correct parties p, q that sent **decide** messages for values $m_p \neq m_q$. Let v_p, v_q be the GA epochs at the conclusion of which the decision was made for p, q respectively. In other words, v_p is the GA epoch for which p output $(m_p, 1, \sigma)$ from CGA at epoch v_p , which we denote by $\text{CGA}[v_p]$, and analogously for v_q . Assume w.l.o.g. that $v_p \leq v_q$. By consistency ([Lemma 5.8](#)) of $\text{CGA}[v_p]$, we get that it can't be that $v_p = v_q$, hence we have that $v_p < v_q$. We now prove the following claim.

Claim 5.15. *For any view v $(v_p, 0) < v \leq (v_q, 0)$, no valid certificate is created for any value $m \neq m_p$ for view v . Furthermore, at most $n - 2f - 1$ correct parties commence view v with $\mathbf{m}(\text{cert}) \neq m_p$.*

Proof. We begin by considering the view $v = (v_p + 1, 0)$. First notice that since p outputs $(m_p, 1)$ in $\text{CGA}[v_p]$, by consistency of the CGA this implies that $f + 1$ correct parties output $(m_p, *)$ from $\text{CGA}[v_p]$ and in particular set cert such that $\mathbf{m}(\text{cert}) = m_p, \mathbf{v}(\text{cert}) = (v_p, 0)$. Denote this set of correct parties by S . Hence, none of the parties in S can commence view v with $\mathbf{m}(\text{cert}) \neq m_p$. This follows from [Claim 5.10](#), and the behavior of the protocol, as no certificate of a lower view than $(v_p, 0)$ triggers a change to cert . Thus, for any value $m' \neq m_p$, at most $n - 2f - 1$ correct parties commence view v with $\mathbf{m}(\text{cert}) = m$, [Claim 5.5](#) then guarantees that no certificate to a value $m' \neq m_p$ can be created during view v . This completes the base case. Note that the above argument, and in particular [Claim 5.5](#) and [Claim 5.12](#), proves the claim for any view $(v_p, 0) < v' \leq (v_q, 0)$ in a leader epoch, conditioned on the claim holding for all views $(v_p, 0) \leq v'' < v$. The proof is thus concluded by induction on the view number. $\square$

As no certificate of view v_q is created for any other value other than m_p , it has to be that $m_q = m_p$. This proves that no two correct parties can send **decide** messages for different values.

Since decision and halting require observing $f + 1$ decide messages, and at least one of those was sent by a correct party, we can conclude from [Lemma 5.14](#) that no two correct parties can observe $f + 1$ decide messages for two different values. This proves that all correct parties that terminate are in agreement. $\square$

Lemma 5.16. *Against any f -bounded adversary, all correct parties output and terminate at most $O(f)$ rounds after GST. In other words, $\mathbf{L}(\text{Lifeline}) = O(f)$.*

Proof. Let GST be as chosen by $\mathcal{A}$. Let t_s be the synchronization time, occurring at most $O(f)$ rounds after GST, guaranteed by Sync and [Theorem 5.3](#). If there exists a correct party p that terminated prior to t_s , then p observed $f + 1$ decide messages for a value m , and multicast a certificate of these $f + 1$ messages to all parties, hence all other correct parties terminate at most Δ time after GST with the same output, due to [Lemma 5.14](#). Otherwise, all correct parties haven't terminated by t_s . Note that since there is no clock drift between correct parties after GST, [Theorem 5.3](#) implies that there exists a synchronization time within every view $v \geq \mathbf{v}(t_s)$ in

which $\text{leader}(v)$ is correct. Let v be the first such view which is not in a CGA epoch, and let t_v be the corresponding synchronization time. Note that such in particular one can choose t_v such that it occurs at most $O(f)$ time after t_s (Since a CGA epoch takes $O(n) = O(f)$ time). We now prove the following claim. We can assume for the rest of the proof, that no correct party terminates during time $[t_v, t_v + T]$, as otherwise all correct parties terminate within Δ time.

Claim 5.17. *Let v be a view in a leader epoch with synchronization time t_v in which $\text{leader}(v)$ is correct, and for which all correct parties do not terminate during view $[t_v, t_v + T]$. Then, there exists a value m s.t. all correct parties have $\mathbf{m}(\text{cert}) = m, \mathbf{v}(\text{cert}) = v$ by time $t_v + T$.*

Proof. We are post GST, and hence the network is synchronous and there is no clock drift between correct parties, and as mentioned in Theorem 5.3, correct parties commence view v at most 2Δ time apart. In particular this implies by the 4Δ waiting time in the behavior of LV, that $\text{leader}(v)$ receives **commence** messages from all correct parties before sending an **opinion** message. This implies that $v_{\max}$ as observed by $\text{leader}(v)$ is at least $\mathbf{v}(\text{cert})$ of all correct parties. This implies that no matter whether $\text{leader}(v)$ sends an **opinion**₁ or **opinion**₂ message for some value m , all correct parties invoke LV.Certification with value m . In turn, again by the 4Δ waiting time on the leader, all correct parties have sufficient time to send all **propose** messages to the leader and hear back about $\mathcal{C}(m, v)$ and , and thus all correct parties set **cert** to $\mathcal{C}(m, v)$. Note that from the perspective of any correct party it takes at most 20Δ time to receive $\mathcal{C}(m, v)$ from $\text{leader}(v)$ in during view v , as in the worst case, a correct party initiated view v 2Δ time prior to the leader, waited 5Δ time to hear an **opinion** message from $\text{leader}(v)$, and waited at most 5Δ time to hear of a level 1 certificate from $\text{leader}(v)$. Thus setting the view length to be $T = 20\Delta$ allows one to claim that all correct processes exit view v with $\text{cert} = \mathcal{C}(m, v)$. $\square$

From this point on the execution is univalent. Formally, as no other certificate of view v for a different value exists, Claim 5.5 guarantees that in all future leader epoch views, no certificate for any other value can be made. This implies that in the next CGA epoch, which occurs at most $O(f)$ time after t_v , all correct parties commence the CGA with input m at most 2Δ time apart(Theorem 5.3). By eventual validity, we have that all correct parties output $(m, 1)$, and thus all correct parties decide m and terminate within Δ time afterwards. For round complexity, note that t_s occurs at most $O(f)$ rounds after GST, t_v occurs at most $O(f)$ rounds after t_s , and by behavior of the protocol, the following CGA after t_v occurs after at most $O(f)$ rounds. The CGA has round complexity $O(f)$, and thus all parties terminate $O(f)$ rounds after GST. $\square$

Lemma 5.18. *Assume all parties are correct and have the same input value m , then all parties output m .*

Proof. We establish that when all parties are correct and all have input m , no party p_i can ever have $\mathbf{m}(\text{cert}_i) \neq m$ (Recall that $\mathbf{m}(\perp)$ is equal to the input value, which is m). We prove this by induction on the views. We begin with the induction step, let v be some view, and assume that for all $v' < v$, no certificate of view v' has been created for any value $m' \neq m$. If v is a CGA view, then by Claim 5.12, no certificate for any value other than m can be created during the CGA, and in particular no party exits the CGA with $\mathbf{m}(\text{cert}) \neq m$. Similarly, if v is leader epoch view, then by Claim 5.5, no certificate for any value other than m can be created during view v , and in particular no party exits the LV execution of view v with $\mathbf{m}(\text{cert}) \neq m$.

The induction base clearly holds due to the fact that at the beginning of time, all parties have $\text{cert} = \perp$, and no certificates for any value $m' \neq m$ exist.

This proves that throughout the execution of the protocol, no certificates for any value other than m are ever created, thus no party ever outputs a value m' with grade 1 from the CGA for any $m' \neq m$. In particular, no party ever decides a value $m' \neq m$. Combined with Lemma 5.16, we conclude that all parties output m , as required. $\square$

All that is left is to analyze the communication complexity of Lifeline. For this purpose, we show the following.

Claim 5.19. *After GST, each correct party sends an "Enter Epoch" message, as per the view synchronizer Sync, for at most $O(1)$ epochs.*

Proof. At GST, let p be a correct party in the "most advanced" epoch, i.e. a correct party that has sent an "Enter epoch" message for the largest epoch number. Let e be that epoch. We have by the behavior of Sync and the Δ bound on message delivery post GST, that by time $GST + 2\Delta$, all correct parties have entered an epoch $e' \geq e$. Moreover, in the time interval $[GST, GST + 2\Delta]$, each correct party has multicast an "Enter epoch" for at most 2 epochs, due to the Δ waiting time in step 4 of Sync.

Next, we observe that $e \leq e' \leq e + 1$. Assume that $e' \geq e + 2$, this implies that by time $GST + 2\Delta$, $n - 2f > 0$ correct parties have multicast an "Epoch completed" message for epoch $e' - 1 \geq e + 1$, which in particular means at least $n - 2f > 0$ correct parties have entered epoch $e' - 1$ at time $GST + 2\Delta - \Omega(f)$, which contradicts the assumption that e is the largest number epoch for which an "Enter epoch" message was sent at GST. Now, we claim that there exists a synchronization time at epoch $e + 1$. Let p' be the first correct party to multicast an "Enter epoch" message for epoch $e + 1$. By our assumption, this message was sent at time $t' > GST$. Thus, by the behavior of Sync, all correct parties enter epoch $e + 1$ by time $t' + 2\Delta$. Furthermore note that since p' is the first correct party to multicast an "Enter epoch" message for epoch $e + 1$, no correct party has spent more time in epoch $e + 1$ than p' by time $t' + 2\Delta$. Thus no correct party spent more than 2Δ time in epoch $e + 1$. Combined, this implies that epoch $e + 1$ has a synchronization time (in all of its views). Theorem 5.3 then concludes the proof since each correct party spends $\Omega(f)$ time in any given epoch, and we proved in Lemma 5.16 that $L(\text{Lifeline}) = O(f)$. $\square$

Lemma 5.20. $CC(\text{Lifeline}) = O(\lambda n^2)$

Proof. Claim 5.19 established that after GST each correct party executes the protocol logic of at most a constant number of epochs, and sends $O(1)$ messages as part of Sync. The proofs of Claim 5.7, Claim 5.13 establish that both in leader epochs and CGA epochs, each correct party multicasts at most $O(\lambda n)$ bits. Thus in total correct parties multicast at most $O(\lambda n^2)$ bits from GST to the timestep in which the last correct party terminates. $\square$

References

- [ADGFT04] Marcos K. Aguilera, Carole Delporte-Gallet, Hugues Fauconnier, and Sam Toueg. Communication-efficient leader election and consensus with limited link synchrony. In *Proceedings of the Twenty-Third Annual ACM Symposium on Principles of Distributed Computing*, PODC '04, page 328–337, New York, NY, USA, 2004. Association for Computing Machinery.

- [BGP92] Piotr Berman, Juan A Garay, and Kenneth J Perry. Bit optimal distributed consensus. In *Computer science: research and applications*, pages 313–321. Springer, 1992.
- [BMR11] Olivier Baldellon, Achour Mostéfaoui, and Michel Raynal. A necessary and sufficient synchrony condition for solving byzantine consensus in symmetric networks. In *International Conference on Distributed Computing and Networking*, pages 215–226. Springer, 2011.
- [BMR15] Zohir Bouzid, Achour Mostfaoui, and Michel Raynal. Minimal synchrony for byzantine consensus. In *Proceedings of the 2015 ACM Symposium on Principles of Distributed Computing*, pages 461–470, 2015.
- [Buc16] Ethan Buchman. *Tendermint: Byzantine fault tolerance in the age of blockchains*. PhD thesis, 2016.
- [CDG⁺22] Pierre Civit, Muhammad Ayaz Dzulfikar, Seth Gilbert, Vincent Gramoli, Rachid Guerraoui, Jovan Komatovic, and Manuel Vidigueira. Byzantine consensus is $\Theta(n^2)$: The dolev-reischuk bound is tight even in partial synchrony! [extended version]. *arXiv preprint arXiv:2208.09262*, 2022.
- [CGG⁺24] Pierre Civit, Seth Gilbert, Rachid Guerraoui, Jovan Komatovic, Anton Paramonov, and Manuel Vidigueira. All byzantine agreement problems are expensive. In *PODC*, pages 157–169. ACM, 2024.
- [DLS88] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the presence of partial synchrony. *Journal of the ACM (JACM)*, 35(2):288–323, 1988.
- [DR85] Danny Dolev and Rüdiger Reischuk. Bounds on information exchange for byzantine agreement. *Journal of the ACM (JACM)*, 32(1):191–204, 1985.
- [DS83] Danny Dolev and H. Raymond Strong. Authenticated algorithms for byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.
- [FLP85] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM (JACM)*, 32(2):374–382, 1985.
- [GAC⁺24] Neil Giridharan, Ittai Abraham, Natacha Crooks, Kartik Nayak, and Ling Ren. Granular synchrony. In *38th International Symposium on Distributed Computing, DISC 2024*, page 30. Schloss Dagstuhl-Leibniz-Zentrum für Informatik GmbH, Dagstuhl Publishing, 2024.
- [HMT07a] Moumen Hamouma, Achour Mostéfaoui, and Gilles Trédan. Byzantine consensus with few synchronous links. In *Principles of Distributed Systems: 11th International Conference, OPODIS 2007, Guadeloupe, French West Indies, December 17-20, 2007. Proceedings 11*, pages 76–89. Springer, 2007.
- [HMT07b] Moumen Hamouma, Achour Mostéfaoui, and Gilles Trédan. Byzantine consensus with few synchronous links. In *OPODIS*, volume 4878 of *Lecture Notes in Computer Science*, pages 76–89. Springer, 2007.

- [LSP82] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. *ACM Trans. Program. Lang. Syst.*, 4(3):382–401, jul 1982.
- [MR21] Atsuki Momose and Ling Ren. Optimal communication complexity of authenticated byzantine agreement. In *DISC*, volume 209 of *LIPICs*, pages 32:1–32:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.
- [Nak08] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.